Universidad Nacional Autónoma de México

Facultad de Ingeniería

Semestre: 2026-2

Materia: Bases de datos

Profesor: Fernando Arreola Franco

Alumno: Pacheco Pacheco Hannia Ivonne

Grupo: 01

Tarea 08

Fecha de entrega: 13 de abril de 2026

Tipos de datos en postgresql

Numéricos:

1. **Tipos enteros:** Almacenan números sin componentes fraccionarias, de diversos rangos. Los intentos de almacenar valores fuera del rango permitido darán como resultado un error.
   a) **Smallint (int2):** Entero de rango pequeño, 2 bytes, rango de -32,768 a +32,767. Generalmente, solo se utiliza cuando el espacio en disco es limitado.
   b) **Integer (int, int4):** El tipo estándar para números enteros, 4 bytes, rango de -2,147,483,648 a +2,147,483,647. Es la opción más común, ya que ofrece el mejor equilibrio entre alcance, capacidad de almacenamiento y rendimiento.
   c) **Bigint (int8):** Entero de rango grande, 8 bytes, rango de -9×10^{18} $a + 9 \times 10^{18}$. Está diseñado para usarse cuando el alcance de integer es insuficiente.

   Los nombres de tipo int2, int4y int8son extensiones, que también son utilizadas por algunos otros sistemas de bases de datos SQL.

   Ejemplo:

   ```
   CREATE TABLE ejemplo_enteros (
      pequeno SMALLINT,
      estandar INTEGER,
      grande BIGINT
   );
   INSERT INTO ejemplo_enteros VALUES (32767, 2147483647, 9223372036854775807);
   ```

2. **Números con precisión arbitraria:** Este tipo de dato puede almacenar números con una gran cantidad de dígitos. Se recomienda especialmente para almacenar cantidades monetarias y otras cantidades que requieran precisión. Los cálculos arrojan resultados exactos siempre que sea posible. Sin embargo, los cálculos son muy lentos en comparación con los tipos enteros o con los tipos de coma flotante.
   a) **Numeric (p, s)**
   b) **Decimal(p, s)**

   Donde p indica el número total de dígitos y s el número de dígitos a la derecha del punto decimal.

   Ejemplo:

   ```
   CREATE TABLE ejemplo_decimales (
      saldo NUMERIC(10,2), -- 10 dígitos totales, 2 decimales
      iva DECIMAL(4,2)     -- Máximo 99.99
   );
   INSERT INTO ejemplo_decimales VALUES (1500.50, 16.00);
   ```

3. **Tipos de punto flotante:** números reales que incluyen una parte fraccionaria o decimal
   a) **Real:** Punto flotante de precisión simple, 4 bytes, aproximadamente 6 dígitos de precisión.
   b) **Double precision:** Punto flotante de doble precisión, 8 bytes, aproximadamente 15 dígitos de precisión**.**

   Ejemplo:

```
CREATE TABLE ejemplo_flotantes (
   temp_clima REAL,            -- 6 dígitos de precisión
   calculo_fisico DOUBLE PRECISION -- 15 dígitos de precisión
);
INSERT INTO ejemplo_flotantes VALUES (25.6, 3.14159265358979);
```

4. **Tipos seriales:** Utilizados comúnmente para claves primarias, ya que son una conveniencia notacional para crear columnas de identificadores únicos
   a) **Smallserial (serial2)**: 2 bytes (rango 1 a 32,767). Los nombres de tipo smallserial y serial2 también funcionan de la misma manera, excepto que crean una smallint columna.
   b) **Serial (serial4)**: 4 bytes (rango 1 a 2,147,483,647). Los nombres de tipo serial y serial4 son equivalentes: ambos crean columnas con datos integer.
   c) **Bigserial (serial8)**: 8 bytes ($rango\ de\ 1\ a\ 9 \times 10^{18}$). Los nombres de tipo bigserial y serial8 funcionan de la misma manera, excepto que crean una bigint columna. Bigserial debe usarse si anticipa el uso de más de 2^{31} identificadores durante la vida útil de la tabla.

   Ejemplo:

```
CREATE TABLE ejemplo_seriales (
   id_corto SMALLSERIAL, -- Hasta 32,767
   id_normal SERIAL,     -- Hasta 2.1 mil millones
   id_gigante BIGSERIAL  -- Para Big Data
);
INSERT INTO ejemplo_seriales DEFAULT VALUES;
```

Carácter: permiten almacenar letras, números y símbolos, con una longitud máxima definida o de hasta 1 GB.

1. **Varchar(n):**
   - Almacena cadenas de longitud variable con un límite máximo de n.
   - Es ideal para campos donde se conoce la longitud máxima, como nombres de usuario o correos electrónicos.
   - Si se intenta guardar una cadena más larga, genera un error, a menos que el exceso sean espacios.
2. **Char(n):**
   - Almacena cadenas de longitud fija n.

- Si la cadena es más corta que n, PostgreSQL la rellena con espacios a la derecha.
- Útil para códigos de longitud constante.

3. **Text**:
   - Almacena cadenas de caracteres de cualquier longitud (variable).
   - No tiene un límite de longitud definido por el usuario (hasta 1 GB por campo).
   - Es el tipo más versátil y recomendado cuando la longitud del texto es desconocida o muy grande.

Ejemplo:

```
CREATE TABLE ejemplo_texto (
   sigla CHAR(3),          -- Siempre ocupa 3 espacios. 'MX' -> 'MX '
   username VARCHAR(20),   -- Hasta 20 caracteres.
   biografia TEXT          -- Sin límite (hasta 1GB)
);
INSERT INTO ejemplo_texto VALUES ('MEX', 'juan_perez', 'Amante de las bases de datos...');
```

Fecha:

1. **Date**: almacena solo la fecha (año, mes, día). Rango: 4713 a.C. hasta 5874897 d.C. Formato AAA-MM-DD
2. **Time [(p)]:** Almacena la hora del día sin zona horaria con formato HH:MM:SS.
3. **Timetz [(p)]:** Almacena la hora del día con zona horaria.
4. **Timestamp [(p)]:** Almacena fecha y hora juntas, sin zona horaria.
5. **Timestamptz [(p)]:** Recomendado para la mayoría de usos. Almacena fecha y hora convertidas a UTC con la zona horaria del servidor.
6. **Interval**: lmacena períodos de tiempo (ej. "1 día", "2 horas", "15 minutos").

Ejemplo:

```
CREATE TABLE ejemplo_fechas (
   solo_fecha DATE,          -- '2024-03-15'
   solo_hora TIME,           -- '14:30:00'
   hora_zona TIMETZ,         -- '14:30:00-05'
   todo TIMESTAMP,           -- '2024-03-15 14:30:00'
   todo_zona TIMESTAMPTZ,    -- '2024-03-15 14:30:00-05' (Recomendado)
   duracion INTERVAL         -- '1 day 2 hours'
);
INSERT INTO ejemplo_fechas VALUES
('2024-12-31', '23:59:59', '23:59:59-06', '2024-12-31 23:59:59', NOW(), '3 months 15 days');
```

Tipo booleano: almacena valores de true, false o null.

Tipo UUID: almacena identificadores únicos universales de 128 bits, representados como 32 dígitos hexadecimales separados por guiones Es más eficiente que usar texto para llaves primarias, ideal para sistemas distribuidos por su baja probabilidad de colisión. Ocupa 16 bytes.

Tipo geométrico: para representar objetos espaciales bidimensionales, incluyendo puntos, líneas, cajas, polígonos y círculos. Estos tipos permiten cálculos geométricos en un plano cartesiano (x, y).

a) **Point**: Puntos geométricos en un plano (x, y).
b) **Line**: Línea infinita
c) **Lseg: (Line Segment)**: Segmento de línea definido por dos puntos finales.
d) **Box:** Rectángulo definido por esquinas superior derecha e inferior izquierda.
e) **Path:** Secuencia de puntos conectados, puede ser abierta o cerrada.
f) **Polygon:** Ruta cerrada o polígono definido por múltiples puntos.
g) **Circle:** Círculo definido por un centro y un radio.

Referencias:

[1] PostgreSQL Global Development Group, "8.1. Numeric Types," PostgreSQL 17 Documentation, 2024. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-numeric.html#DATATYPE-NUMERIC-DECIMAL

[2] PostgreSQL Global Development Group, "8.3. Character Types," PostgreSQL 17 Documentation, 2024. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-character.html

[3] PostgreSQL Global Development Group, "8.5. Date/Time Types," PostgreSQL 8.2 Documentation, 2006. [En línea]. Disponible en: https://www.postgresql.org/docs/8.2/datatype-datetime.html

[4] PostgreSQL Global Development Group, "8.8. Geometric Types," PostgreSQL 17 Documentation, 2024. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-geometric.html

[5] Tutoriales Programación Ya, "Tipos de datos básicos (boolean)," PostgreSQL Ya. [En línea]. Disponible en: https://www.tutorialesprogramacionya.com/postgresqlya/temarios/descripcion.php?cod=175&punto=17&inicio=

[6] Neon, "PostgreSQL UUID," Neon Documentation, 2024. [En línea]. Disponible en: https://neon.com/postgresql/postgresql-tutorial/postgresql-uuid